LELEC2885 - Image Processing
Project Guidelines

Academic year 2025-26

Edouard Couplet (edouard.couplet@uclouvain.be)
Benoit Gérin (benoit.gerin@uclouvain.be)
Mathieu Reniers (mathieu.reniers@uclouvain.be)

Do not be afraid by the length of this document. The aim is twofold: to explain what is expected in the project and to provide you explanations on more specific points that you may not have seen during lectures. Do not read everything in detail in one go. Instead, consult this document for help when you need it.

Contents

1 Context of the project and general guidelines **2**
1.1 The LLNDataset . . . 2
1.2 Schedule and Deliverable . . . 4
1.3 Content of the Report . . . 4

2 PART A. A classic supervised segmentation task **5**
2.1 Towards a first run of the code . . . 5
2.1.1 Pytorch library . . . 5
2.1.2 Albumentations library . . . 6
2.1.3 Other libraries . . . 6
2.1.4 Code structure . . . 6
2.2 Train a basic network . . . 7
2.3 Implement your own architecture . . . 7

3 PART B. A dive into self-supervised learning **8**

---

1 Context of the project and general guidelines

In this project, you will implement a semantic segmentation pipeline to classify land use from aerial imagery. Land use mapping is a fundamental task in urban planning, environmental monitoring, agriculture, and disaster management. The core idea is to assign each pixel in an image to a land cover category (e.g., water, forest, urban, agricultural, industrial, roads, etc.). This classification enables large-scale, automated analysis and can provide valuable insight into how landscapes are organized and how they evolve over time.

The project is divided into two parts:

PART A. A classic supervised segmentation task. For this first part, you will be asked to implement a deep learning method to generate segmentation masks from aerial imagery of Louvain-la-Neuve. For this purpose, a workflow is provided based on the Pytorch library. To achieve this task successfully, you will need (at least) to implement your own network architecture, implement the training loop and enhance the evaluation procedure. This first part of the project should be completed by week 10 and be presented to the teaching assistants during an intermediate oral presentation of 10 minutes.

PART B. A dive into self-supervised learning. For this second part, you will be asked to explore a self-supervised learning approach to semantic segmentation, based on the work done in part A. This second part is due by week 13.

1.1 The LLNDataset

In this project, we will be working with a custom dataset: the LLNDataset. It is built from an aerial view of Louvain-la-Neuve captured during the walloon 2022 spring orthophoto campaign. The considered image is 4096 by 3072 pixels, and corresponds to an area of 4096×3072 meter, such that one pixel is precisely one meter. Only RGB channels were extracted (no infra-red channel).

The masks, or segmentation labels, were extracted from OpenStreetMap (OSM) data. We define five land-use categories: *buildings*, *water*, *green spaces*, *farmlands*, and *others*. Each category groups one or several OpenStreetMap tags, as detailed in Table 1. The aerial view and corresponding segmentation masks are shown in Figure 1.

Category	**OpenStreetMap Tags**	**Label**
Water	`natural = water` `landuse = reservoir`	1
Buildings	`building = *`	2
Farmlands	`landuse = farmland`	3

Green Spaces	`landuse = {forest, meadow}`
	`natural = {wood, grassland, scrub}`
	`leisure = {park, garden}`
		4

Others	Remaining unmapped areas	0

Table 1: Mapping between land-use categories and corresponding OpenStreetMap tags.

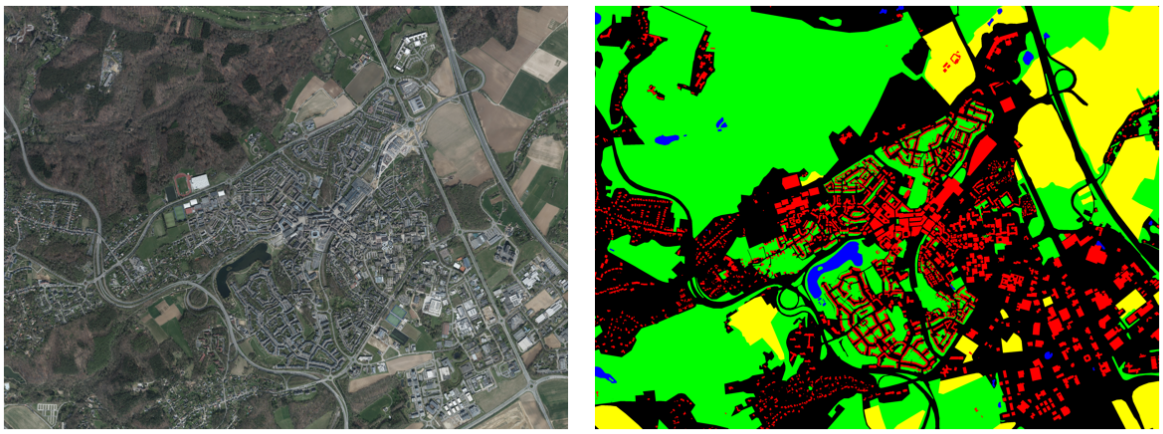


Figure 1: Aerial view of Louvain-la-Neuve and corresponding segmentation masks.

Note that while we use OSM data as our ground truth, it remains only a crude approximation of the reality. First, OSM data is updated continuously and therefore does not accurately reflect the state of Louvain-la-Neuve as it was in 2022 (for instance, some buildings may have been constructed or modified since then). Second, the dataset may contain omissions or inaccuracies due to human error or incomplete mapping. Nevertheless, the data does not need to be perfectly accurate to provide a meaningful learning signal for land-use classification tasks.

To facilitate the learning process, we divide the aerial image and its corresponding masks into tiles of size 64×64 pixels. Tiles are named `tile_RR_CC` where RR and CC indicate row and column indices, zero-indexed from the upper-left corner of the image. The tiles located in the lower-right quadrant of the image are reserved as the test set. The remaining tiles are randomly split into training and validation sets, with proportions of 80% and 20%, respectively. The resulting dataset partitions are illustrated in Figure 2.

In the first part of the project, your objective will be to design and train a deep neural network using the training and validation tiles. Once the model has been trained, you will use it to predict segmentation masks for the test tiles. By assembling these predicted tiles, you can reconstruct the mask for the lower-right quadrant of the original image and compare it to the ground truth, both visually and through appropriate quantitative evaluation metrics. In the second part of the project, you will explore self-supervised approaches to predict meaningful land-use categories without relying on the provided segmentation labels.

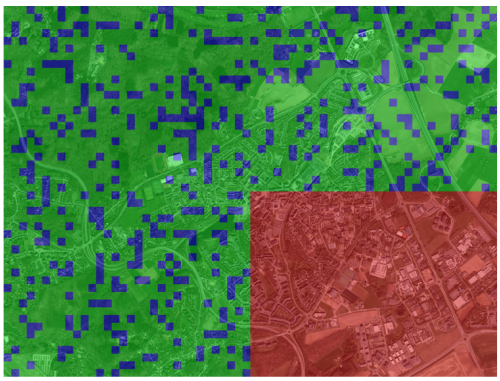


Figure 2: Dataset partitions: train (green), validation (blue), test (red).

1.2 Schedule and Deliverable

The deadline for submitting the project is Wednesday 10 December at 10 p.m. Even if this deadline seems a long way off, do not wait until the last minute to get started. Training a model can take several hours, depending on your computer.

Multiple Q&A sessions will be organised with teaching assistants. Also, a short (mandatory!) intermediate oral presentation of 5 minutes followed by a discussion of 5 minutes will be organised during week 10. Details will follow on Moodle in due time.

This project can be carried out by group of two or three students. Each group is asked to submit on Moodle:

- A zip file with your code (don't re-upload the dataset!);
- A written report. Further instructions are given in section 1.3.

1.3 Content of the Report

Consider that the person who is going to read your report already has a technical background in deep learning. There is no need to explain what is a network, a convolution or a train/validation/test set. The report is there to justify the choices you have made and present your results.

In a pdf report of max. 3 pages (excluding images and references), comment on the following aspects of your project:

PART A. A classic supervised segmentation task.

1. **Network Architecture.**
   Describe **briefly** the network used and justify your design choices. Mention main components and any modifications if based on a known architecture.
2. **Network Training.**
   State the loss function and key hyperparameters (learning rate, epochs, batch size, optimizer, etc.). Include learning curves and comment on overfitting or underfitting and mitigation strategies.
3. **Evaluation.**
   Show visual results (including successes and failures) and quantify performance with suitable metric(s). Discuss insights, limitations, or patterns observed.

PART B. A dive into self-supervised learning.
No specific guidelines are provided for this part of the written report. See details in Section 3.

2 PART A. A classic supervised segmentation task

2.1 Towards a first run of the code

The project is designed to be directly runnable on your computer, provided that all required libraries are installed. A basic network and a dummy training loop are included for you in the `basicNetwork.py` and `model.py`file. After running the `main.py` file, you should be able to view the results in the `Results > DefaultExp > test` folder. Running this initial experiment allows you to verify that the required libraries are installed correctly.

To ease installation, we recommend starting from a clean Python virtual environment. Virtual environments can be created using `conda`, or with `venv` if you are using `pip`. They are isolated from the base environment, preventing conflicts with other projects. We suggest creating a virtual environment with Python 3.12 rather than the latest 3.13, as some dependencies may not yet be fully compatible with the newest Python version. In Sections 2.1.1 and 2.1.2, we provide information about the Pytorch and Albumentations libraries, respectively.

How to launch an experiment?
In the terminal, run the following command[a]:
$ python main.py -exp DefaultExp
The exp argument defines the the name of the yaml file (in the `Todo_list` folder) with the wanted configuration (here `DefaultExp.yaml`)

[a] If you receive a lot of warnings, use this command:
$ python -W ignore main.py -exp DefaultExp

2.1.1 Pytorch library

In this project, you will use the PyTorch library to implement your deep learning framework. PyTorch is well-documented, and a variety of tutorials are available to help you get started. Installation instructions can be found here. Select your operating system (Linux, macOS, or Windows) and choose the appropriate Compute Platform: "CPU" or "CUDA." The corresponding command will be displayed on the PyTorch installation page.

PyTorch can run on either CPU or GPU. Installing PyTorch for CPU is generally straightforward and the project can be completed successfully using only on a CPU. Installing PyTorch with GPU support can be longer and occasionally complex, but it allows your code to run significantly faster. To run PyTorch on a GPU, your computer must have a compatible GPU, and the necessary drivers must be installed. CUDA is the driver framework that enables parallel computation on NVIDIA GPUs.

Can your computer support PyTorch on GPU?
Not all GPUs are compatible with CUDA. To check whether your computer has a CUDA-compatible GPU, run the following commands in your Python environment:
$ import torch
$ print(torch.cuda.is_available())

If the output is `True`, your system has a compatible GPU and PyTorch can run on it. If the output is `False`, only the CPU will be available to run the code.

In our project, you can choose between of the use of the CPU or the GPU by changing the `DEVICE` parameter in the `Todo_List > DefaultExp.yaml` file.

2.1.2 Albumentations library

The second deep learning-dedicated library to install is Albumentations. This library will be used to perform the data augmentation process. Do not worry, the installation is straightforward and is not specific to the use of the CPU or the GPU.

2.1.3 Other libraries

You will need to install termcolor. We also recommend you to use the Lovely tensors library to print information on your pyTorch tensors.

2.1.4 Code structure

It is common for a deep learning framework to require a larger amount of code than a "classical" computer vision project. If no particular attention is paid to it, the code can look messy and may not be practical to use. To help you, we already provide you with a project structure that improves the code modularity. The objective is to run multiple experiments by launching one unique main function with different configuration files (yaml files in the `Todo_List` folder).

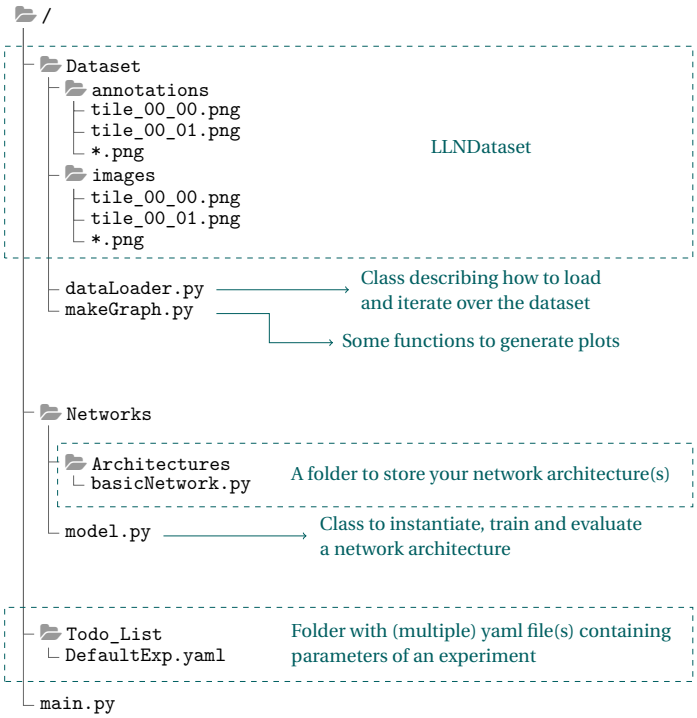


Figure 3: Structure of the provided code.

The code structure is presented in Figure 3. There are 3 folders in the root directory:

1. **Dataset.** This folder contains functions used to download the dataset, iterate over the train/-validation/test sets and show some images.
2. **Networks.** This folder contains functions used to defined the network architecture + train and evaluate it.
3. **Todo_List.** This folders can contain as many `yaml` files as you want. Each one will correspond to one set of hyperparameter to test.

2.2 Train a basic network

To start and have a first baseline with a real training procedure, you should complete the missing parts of the code as described in the box below.

You are expected to complete the following code sections in the `model.py file`:

1. **`__init__` function**
   You should complete :
   - line 66 to add the loss function
   - line 67 to add the optimizer

   that will be considered for training your network in the segmentation task.
2. **`train` function**
   You must write the loop to train and validate your model for a given number of epoch. At the end of the training, you are asked to record train and validation loss curves into a graph.
3. **`evaluate` function**
   We have provided code for qualitative evaluation (visual inspection). We ask you to complete the `evaluate` function with quantitative evaluation: identify one or more suitable metrics to quantify the performance of your model.

2.3 Implement your own architecture

Now that you have implemented a working training loop and proper evaluation procedure, it is time to improve upon the basic network architecture provided in the `basicNetwork.py` file.

You are asked to implement an *encoder-decoder* style architecture. This type of newtork is widely used for dense prediction tasks such as semantic segmentation, where the goal is to assign a particular value to each pixel in an image (in this case, land-use class labels). The encoder progressively reduces the spatial resolution of the input while increasing the number of feature channels. It compresses the input image into a (hopefully) rich and compact representation. The decoder then reconstructs a high-resolution output (in this case, segmentation masks) from this compressed representation.

Concretely, your encoder should take as input a batch of RGB tiles of size $(B, 3, H, W)$ and produce an intermediate representation of size (B, C, H', W'), where B is the batch size, (H, W) the height and width of input tiles, C the number of channels in the representation, with $C > 3$ and $H', W' < H, W$. The decoder should then take this representation as input and output a tensor of size $(B, 5, H, W)$. Each of the five output channels corresponds to one of the five land-use categories. These outputs are typically either *unnormalized logits* (the raw outputs of the last layer) or *normalized probabilities* after applying a softmax function, depending on your implementation. For evaluation and visualization, you can aggregate the channels into a single segmentation mask (e.g, by taking the `argmax` over the channel dimension). Your encoder–decoder design should also preserve spatial coherence. In other words, a feature located in the upper-right region of the encoded representation should correspond to information aggregated from the upper-right region of the original input tile.

Create a new python file under the `Architectures` folder and implement an encoder-decoder style architecture that improves upon the provided `basicNetwork.py`. You do not need to reinvent the wheel: do not hesitate to adapt existing architectures from the literature to your use case. If so, do not forget to mention your sources in the report.

3 PART B. A dive into self-supervised learning

In the second part of the project, you will reuse the encoder–decoder architecture developed in Part A, but this time you will train your model **without using any segmentation labels**. Instead, you will design a *self-supervised learning* (SSL) procedure based on a **proxy task**—a task for which supervision can be derived directly from the input data itself. This may require minor adaptations to your architecture (typically in the output layers), depending on the nature of the chosen proxy task. The objective is to identify a suitable proxy task that enables the encoder to learn meaningful representations of the data without relying on explicit human annotations (in this case, OpenStreetMap data).

Once your network has been trained on the proxy task, you will fit a **clustering algorithm** to the feature representations learned by the encoder. Recall that the encoder outputs feature maps of size (C, H', W'), where each element (or "pixel") in this feature space corresponds to a spatial region of the original input tile of size (H, W). To perform clustering, first gather all encoded feature vectors from the tiles in the training set into a single array of size (N, C), where $N = N_{\text{train}} \times H' \times W'$, with N_{train} being the number of tiles in the training set. You can then apply the clustering method of your choice on the feature vectors. The number of clusters and specific algorithm are left to your decision—feel free to experiment with different configurations.

Once your clustering algorithm is fitted to the training data, encode the tiles from the testing data and assign each encoded "pixel" to its corresponding cluster. Reshape the resulting labels back into 2D grids of size (H', W'). Next, upscale these grids to the original tile resolution (H, W) and stitch tiles back together to obtain a **pseudo-segmentation mask**, where each region corresponds to one of the discovered clusters. Finally, visualize this map and compare to the ground-truth land-use image/mask. Do some of the discovered clusters correspond to the predefined land-use classes (*e.g.*, buildings, water, farmlands, etc.)? Do you observe new, unexpected categories emerging from the clustering?

The first part of the project is worth 12 out of 20 points, while the second part is worth 8 out of 20 points.

For the code of Part B, we recommend copying the entire project folder so that you do not lose any work from Part A and can keep the two parts well separated.

There is no specific structure to follow for the written report of Part B. You should clearly specify the proxy task, any adaptations made to your architecture, and the clustering algorithm you used. In addition, include any analyses you consider relevant and summarize your main conclusions.